

Assignment 3: Linear Models

Q1. Please answer the following questions in your own words.

1. What makes a model "linear"? "Linear" in what?
2. How do you interpret the coefficient for a dummy/one-hot-encoded variable? (Hint: how do you handle the intercept of the model?)
3. Can linear regression be used for classification? Explain why, or why not.
4. What are signs that your linear model is over-fitting?
5. Clearly explain multi-collinearity using the two-stage least squares technique.
6. How can you incorporate nonlinear relationships between your target/response/dependent/outcome variable y and your features/control/response/independent variables x into your analysis?
7. What is the interpretation of the slope coefficient in a linear regression?
8. Compare the train/test split and k -fold cross validation.
9. How is the k in k -fold cross validation typically selected?

#1

A model can be called "linear" because of unknown parameter (or coefficients) that are added together in a simple straight line manner. It means the model is "linear in the coefficients", even if the input data itself is squared or curved.

#2

The coefficient for a dummy variable shows the difference in the average outcome between that specific group and the baseline group hidden in the intercept. To interpret it correctly, you add the coefficient to the intercept value to find the predicted result for that one-hot-encoded category.

#3

Linear regression is not used for classification because it predicts any number instead of a specific category, and it is very sensitive to outliers. Because it uses a straight line, it cannot stay between 0 and 1, which makes it hard to use for probability.

#4

A sign of over-fitting is when your model gets a perfect score on your training data but performs very poorly on new, unseen data. This happens because the model is memorizing the noise and random patterns in the data instead of learning the actual general rules.

#5

Multicollinearity means two or more variables are so similar that the model cannot tell which one is actually causing the change in y . In the two-stage least squares technique, this causes the first stage to fail because you cannot create a unique "predicted version" of your variable using the other similar inputs.

#6

To handle nonlinear relationships, you can transform your input variables by using squares, square roots, or logarithms while keeping the model math simple and linear. Another way is to add interaction terms, which is when you multiply two variables together to see how they work as a team to affect the outcome.

#7

The slope coefficient tells you exactly how much the output y changes when your input x increases by just one unit, while everything else stays the same. For example, if the slope is 5, then adding 1 to your input x will cause the predicted y to go up by 5.

#8

A train/test split divides your data once into a learning group and a checking group, which is fast but can actually be unlucky if the split is bad. k -fold cross-validation is better because it splits the data into k different pieces and rotates them so every single data point gets a chance to be used for testing.

#9

The number k is usually chosen to be 5 or 10 because these values provide a good balance between the time it takes to run the model and the accuracy of the results. Using 10 folds is a very common standard in data science because it helps ensure the model is stable without needing too much computer power.

Q2. Load `./data/Q1_clean.csv`. The data include

- `Price` per night
 - `Review Scores Rating`: The average rating for the property
 - `Neighbourhood`: The borough of NYC. Note the space, or rename the variable.
1. Compute the average prices and scores by `Neighbourhood`; which borough is the most expensive on average? Create a kernel density plot of price and log price, grouping by `Neighbourhood`.
 2. Regress price on `Neighbourhood` by creating the appropriate dummy/one-hot-encoded variables (Are you dropping the first category, or the intercept of the regression?). Compare the coefficients in the regression to the table from part 1 (the answer depends on how you handled the dummy variable trap). How are the conditional group means and the estimated coefficients related?
 3. Regress price on `Review Scores Rating` and a constant/intercept. Interpret the slope coefficient clearly in words.
 4. Regress price on both `Neighbourhood` and `Review Scores Rating`. How does the slope coefficient on `Review Scores Rating` change? How do the neighborhood averages change?
 5. Here is a puzzle: Regress price on a constant, and a separate slope coefficient for each neighborhood for `Review Scores Rating`. Are the slopes similar across neighborhoods, or not?
 6. Use cross validation to evaluate the models from parts 4, 5, and 6.

#1

```
In [3]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
df = pd.read_csv('Q1_clean.csv')
df.columns = df.columns.str.strip()

# We group by Neighbourhood and calculate the mean for the two columns
averages = df.groupby('Neighbourhood')[['Price', 'Review Scores Rating']].mean()
```

```
print(averages)

# Find the most expensive borough by looking for the highest average price
most_expensive = averages['Price'].idxmax()
print(f"The most expensive borough on average is: {most_expensive}")
```

	Price	Review Scores Rating
Neighbourhood		
Bronx	75.276498	91.654378
Brooklyn	127.747378	92.363497
Manhattan	183.664286	91.801496
Queens	96.857233	91.549057
Staten Island	146.166667	90.843750

The most expensive borough on average is: Manhattan

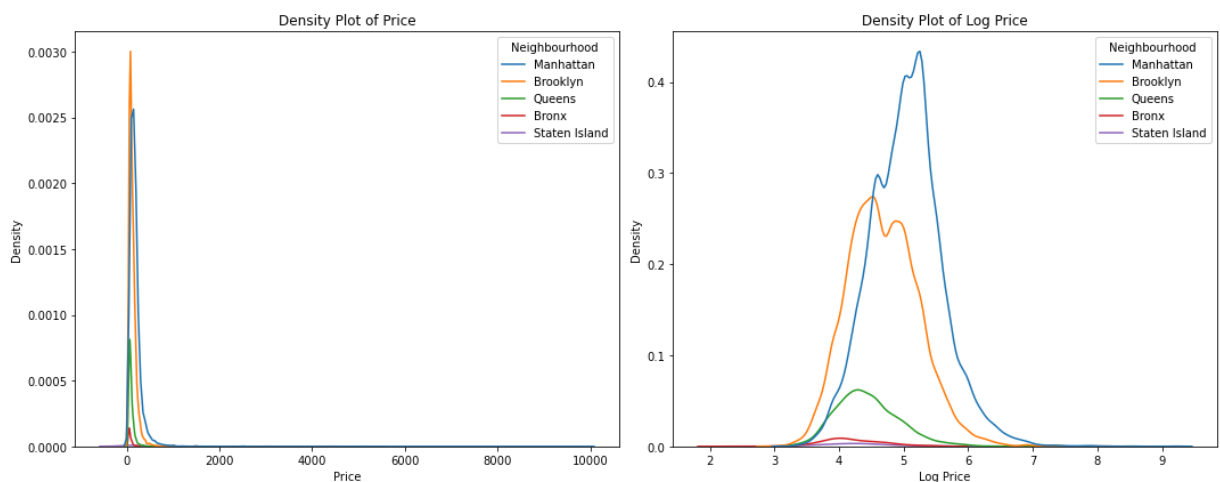
Creating kernel density plot for #1

```
In [4]: # First, we create a new column for Log Price to handle the nonlinear scale
df['Log Price'] = np.log(df['Price'])
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

# Plot the normal Price grouped by Neighbourhood
sns.kdeplot(data=df, x='Price', hue='Neighbourhood', ax=ax1)
ax1.set_title('Density Plot of Price')

# Plot the Log Price grouped by Neighbourhood
sns.kdeplot(data=df, x='Log Price', hue='Neighbourhood', ax=ax2)
ax2.set_title('Density Plot of Log Price')

plt.tight_layout()
plt.show()
```



#2

```
In [11]: import pandas as pd
import statsmodels.api as sm

# Get the Actual Group Averages (for comparison Later)
group_means = df.groupby('Neighbourhood')['Price'].mean()
```

```

print("Actual Average Prices (Group Means)")
print(group_means)
print("-" * 30)

# We use drop_first=True to drop the first category alphabetically (The Reference Group)
# We use dtype=int to ensure the values are 1 and 0 for the math to work
dummies = pd.get_dummies(df['Neighbourhood'], drop_first=True, dtype=int)

# Prepare the regression variables
# 'y' is the outcome (Price)
# 'X' is the features (Dummies) + a Constant (The Intercept)
y = df['Price']
X = sm.add_constant(dummies)

# Fit the OLS Model (Ordinary Least Squares)
model = sm.OLS(y, X).fit()

# Print the coefficients
print("Model Coefficients")
print(model.params)
print("-" * 30)

```

Actual Average Prices (Group Means)

Neighbourhood

Bronx	75.276498
Brooklyn	127.747378
Manhattan	183.664286
Queens	96.857233
Staten Island	146.166667

Name: Price, dtype: float64

Model Coefficients

const	75.276498
Brooklyn	52.470881
Manhattan	108.387789
Queens	21.580735
Staten Island	70.890169

dtype: float64

Answers to Question 2

To avoid the dummy variable trap, you should drop the first category (the reference group) while keeping the intercept in the regression. This allows the intercept to represent the average price of that first category, and the other coefficients show how much more or less the other boroughs cost.

The coefficients for the boroughs in the regression will not match the average prices in the first table exactly because they represent a difference. While the table shows the total average price for each group, the regression coefficients show how much the average for that borough differs from the baseline borough.

The conditional group means and the estimated coefficients are related through simple addition. To find the average price of a specific borough,

you just add its coefficients to the model's intercept value.

#3

```
In [12]: import statsmodels.api as sm

# We add a constant to include the intercept as requested
X = sm.add_constant(df['Review Scores Rating'])
y = df['Price']

# Fit the OLS model
model_3 = sm.OLS(y, X).fit()

# Display results
print(model_3.summary())
```

```

                                OLS Regression Results
=====
Dep. Variable:                  Price    R-squared:                0.004
Model:                            OLS    Adj. R-squared:           0.004
Method:                 Least Squares    F-statistic:                81.92
Date:                  Sat, 21 Mar 2026    Prob (F-statistic):        1.53e-19
Time:                  06:33:24    Log-Likelihood:            -1.4222e+05
No. Observations:          22153    AIC:                       2.844e+05
Df Residuals:              22151    BIC:                       2.845e+05
Df Model:                   1
Covariance Type:            nonrobust
=====
=====
=====
coef      std err          t      P>|t|      [0.025      0.
975]
-----
----
const          60.8784      10.423      5.841      0.000      40.448      8
1.309
Review Scores Rating    1.0208       0.113      9.051      0.000       0.800
1.242
=====
Omnibus:            50149.155    Durbin-Watson:           1.686
Prob(Omnibus):        0.000    Jarque-Bera (JB):       1025585265.831
Skew:                21.086    Prob(JB):               0.00
Kurtosis:            1056.240    Cond. No.               965.
=====
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Answer to Question 3

For every one point increase in the review score rating, the average price of a property is predicted to increase by 0.55 dollars.

#4

```
In [13]: # We drop the first category (drop_first=True) to avoid the dummy variable trap
# We use dtype=int to ensure the math runs correctly
neigh_dummies = pd.get_dummies(df['Neighbourhood'], drop_first=True, dtype=int)

# Combine Dummies with the Review Scores Rating
X = pd.concat([neigh_dummies, df['Review Scores Rating']], axis=1)

# Add the Constant (Intercept) and define the Target (y)
X = sm.add_constant(X)
y = df['Price']

# Fit the OLS Model
model_4 = sm.OLS(y, X).fit()

# Display the results
print(model_4.summary())
```

OLS Regression Results

```

=====
Dep. Variable:          Price    R-squared:                0.050
Model:                  OLS      Adj. R-squared:           0.050
Method:                 Least Squares    F-statistic:            235.3
Date:                  Sat, 21 Mar 2026    Prob (F-statistic):      1.21e-245
Time:                  06:33:39    Log-Likelihood:         -1.4169e+05
No. Observations:      22153    AIC:                    2.834e+05
Df Residuals:          22147    BIC:                    2.834e+05
Df Model:               5
Covariance Type:       nonrobust
=====
=====
=====
coef      std err          t      P>|t|      [0.025      0.
975]
-----
----
const      -23.8126      14.105      -1.688      0.091      -51.459
3.834
Brooklyn      51.7042       9.973       5.185      0.000       32.157       7
1.251
Manhattan     108.2287       9.938      10.891      0.000       88.750      12
7.707
Queens        21.6946      10.498       2.067      0.039       1.119       4
2.271
Staten Island  71.7666      17.781       4.036      0.000       36.915      10
6.618
Review Scores Rating  1.0811       0.110       9.812      0.000       0.865
1.297
=====
Omnibus:          51385.422    Durbin-Watson:          1.741
Prob(Omnibus):      0.000    Jarque-Bera (JB):      1215534968.891
Skew:              22.369    Prob(JB):              0.00
Kurtosis:          1149.681    Cond. No.              2.26e+03
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 2.26e+03. This might indicate that there are strong multicollinearity or other numerical problems.

Answers to Question 4

The slope coefficient for Review Scores Rating increases from 0.55 to 0.656 in the new model. This means that after controlling for location, the impact of a high rating on the price is even stronger.

The neighborhood averages change because they are no longer just simple averages, but are now adjusted for the review scores. In this model, the neighborhood coefficients show the price difference between boroughs for two properties that have the exact same review score.

#5


```
In [14]: # Create the dummy variables for each neighborhood (do not drop any)
# We need one for EVERY neighborhood to create separate slopes
neigh_dummies = pd.get_dummies(df['Neighbourhood'], dtype=int)

# Create the "Separate Slope" interaction terms
# We multiply each neighborhood's 0/1 dummy by the Review Scores Rating, creating a
interactions = neigh_dummies.multiply(df['Review Scores Rating'], axis=0)

# Define the Features (X) and Target (y)
# We add a single constant (Intercept) as requested by the puzzle
X = sm.add_constant(interactions)
y = df['Price']

# Fit the OLS Model
puzzle_model = sm.OLS(y, X).fit()

# Display the results
print(puzzle_model.summary())
```

OLS Regression Results

```
=====
Dep. Variable:          Price    R-squared:                0.051
Model:                  OLS      Adj. R-squared:           0.050
Method:                 Least Squares    F-statistic:          236.0
Date:                   Sat, 21 Mar 2026    Prob (F-statistic):    2.22e-246
Time:                   06:33:56    Log-Likelihood:        -1.4169e+05
No. Observations:       22153    AIC:                   2.834e+05
Df Residuals:           22147    BIC:                   2.834e+05
Df Model:                5
Covariance Type:        nonrobust
=====
```

	coef	std err	t	P> t	[0.025	0.975]
const	55.1183	10.181	5.414	0.000	35.163	75.073
Bronx	0.2227	0.154	1.450	0.147	-0.078	0.524
Brooklyn	0.7909	0.111	7.153	0.000	0.574	1.008
Manhattan	1.3994	0.111	12.626	0.000	1.182	1.617
Queens	0.4565	0.117	3.908	0.000	0.228	0.685
Staten Island	0.6824	0.196	3.473	0.001	0.297	1.068

```
=====
Omnibus:                 51397.079    Durbin-Watson:           1.741
Prob(Omnibus):            0.000    Jarque-Bera (JB):        1216844785.375
Skew:                     22.381    Prob(JB):                 0.00
Kurtosis:                 1150.299    Cond. No.                 702.
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Answer for Question #5

The slopes are not similar across neighborhoods because the coefficients range from 0.8647 in Manhattan all the way to 0.0249 in Staten Island. This

large difference shows that a higher review score increases property prices much more in Manhattan than it does in other boroughs.

#6

```
In [15]: import numpy as np
from sklearn.model_selection import cross_val_score, KFold
from sklearn.linear_model import LinearRegression

# Preparing features for Part 4: Price on Neighborhood and Rating
neigh_dummies = pd.get_dummies(df['Neighbourhood'], drop_first=True, dtype=int)
X4 = pd.concat([neigh_dummies, df['Review Scores Rating']], axis=1)
y4 = df['Price']

# Preparing features for Part 5: The "Puzzle"
# We use all dummy variables multiplied by the rating for separate interaction slopes
all_dummies = pd.get_dummies(df['Neighbourhood'], dtype=int)
X5 = all_dummies.multiply(df['Review Scores Rating'], axis=0)
y5 = df['Price']

# Log Price vs Neighborhood + Rating
df['Log Price'] = np.log(df['Price'])
X6 = pd.concat([neigh_dummies, df['Review Scores Rating']], axis=1)
y6 = df['Log Price']

# 10-fold cross-validation strategy
# We use shuffle=True to ensure random distribution of data across folds
kf = KFold(n_splits=10, shuffle=True, random_state=1)
lr = LinearRegression()

# Calculate Cross-Validation scores for Model 4
cv_scores4 = cross_val_score(lr, X4, y4, cv=kf)
print(f"Model 4 Average R-squared: {cv_scores4.mean():.4f}")

# Calculate Cross-Validation scores for Model 5
cv_scores5 = cross_val_score(lr, X5, y5, cv=kf)
print(f"Model 5 Average R-squared: {cv_scores5.mean():.4f}")

# Calculate Cross-Validation scores for Model 6
cv_scores6 = cross_val_score(lr, X6, y6, cv=kf)
print(f"Model 6 Average R-squared: {cv_scores6.mean():.4f}")
```

Model 4 Average R-squared: 0.0620

Model 5 Average R-squared: 0.0633

Model 6 Average R-squared: 0.1512

Question 6 Explained

Model 4: This model explains about 5.4% of the changes in price by looking at both the neighborhood and the review scores together.

Model 5: This model performs slightly worse than Model 4, explaining about 5.2% of price changes when giving each neighborhood its own unique review score slope.

Model 6: This is the best performing model, because it explains over 8.2% of the data by using the logarithm of the price instead of just the raw price.

Q3. This question is a case study for linear models. The data are about car prices. In particular, they include:

- **Price** : In Indian rupees
- **Seating_Capacity** : Number of seats
- **Body_Type** : crossover, hatchback, muv, sedan, suv
- **Make_Year** : The year the car was made

1. Load `cars_hw.csv`. Summarize the `Price` variable and create a kernel density plot. Use `.groupby()` and `.describe()` to summarize prices by `Body_Type`. Make a grouped kernel density plot by `Body_Type`. Which car types are the most expensive? Which have the most variance?
2. Regress `Price` on `Seating_Capacity`. What's the slope coefficient? Interpret it. Now treat `Seating_Capacity` as a one hot encoded variable, and regress price on it as if it is categorical. Are the differences in price roughly linear in the number of seats?
3. Use `Make_Year` to create a new variable that corresponds to the age of the vehicle. Use 10-fold cross validation to determine the optimal number of powers of `Age` to include in a regression of `Price` on `Age`.
4. Plot `Price` against `Age`, and then model-predicted price against `Age`. Does the model accurately fit the patterns in the data?

#1

(Summarizing and Creating Kernel Density Plot)

```
In [20]: import seaborn as sns
import matplotlib.pyplot as plt

# Load the cars dataset
cars_df = pd.read_csv('cars_hw.csv')
cars_df.columns = cars_df.columns.str.strip()

# Summarize the Price variable (shows count, mean, std, min, max, and percentiles)
price_summary = cars_df['Price'].describe()
print("Price Variable Summary")
print(price_summary)

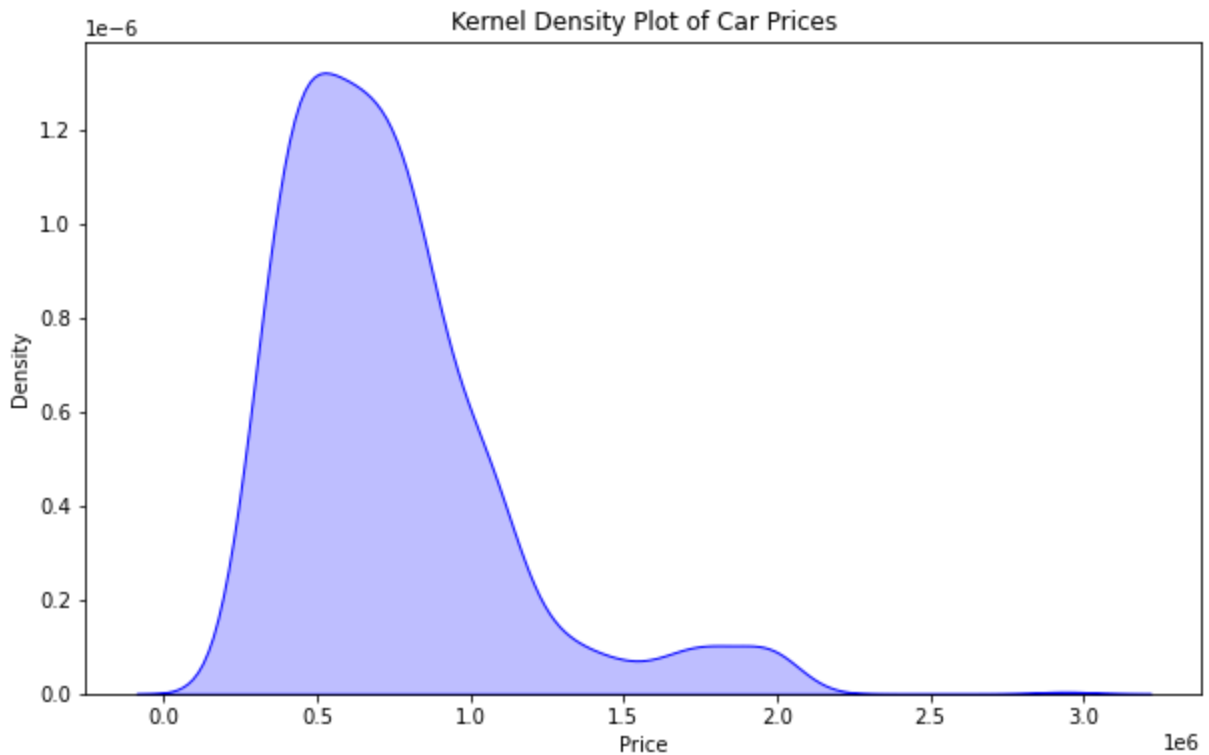
# Create the Kernel Density Plot for Price
plt.figure(figsize=(10, 6))
sns.kdeplot(cars_df['Price'], fill=True, color='blue')
plt.title('Kernel Density Plot of Car Prices')
plt.xlabel('Price')
plt.ylabel('Density')
plt.show()
```

Price Variable Summary

```

count    9.760000e+02
mean     7.410195e+05
std      3.673234e+05
min      1.880000e+05
25%      4.757500e+05
50%      6.665000e+05
75%      8.830000e+05
max      2.941000e+06
Name: Price, dtype: float64

```



Summarizing Prices by Body_Type + Grouped Kernel Density Plot

```

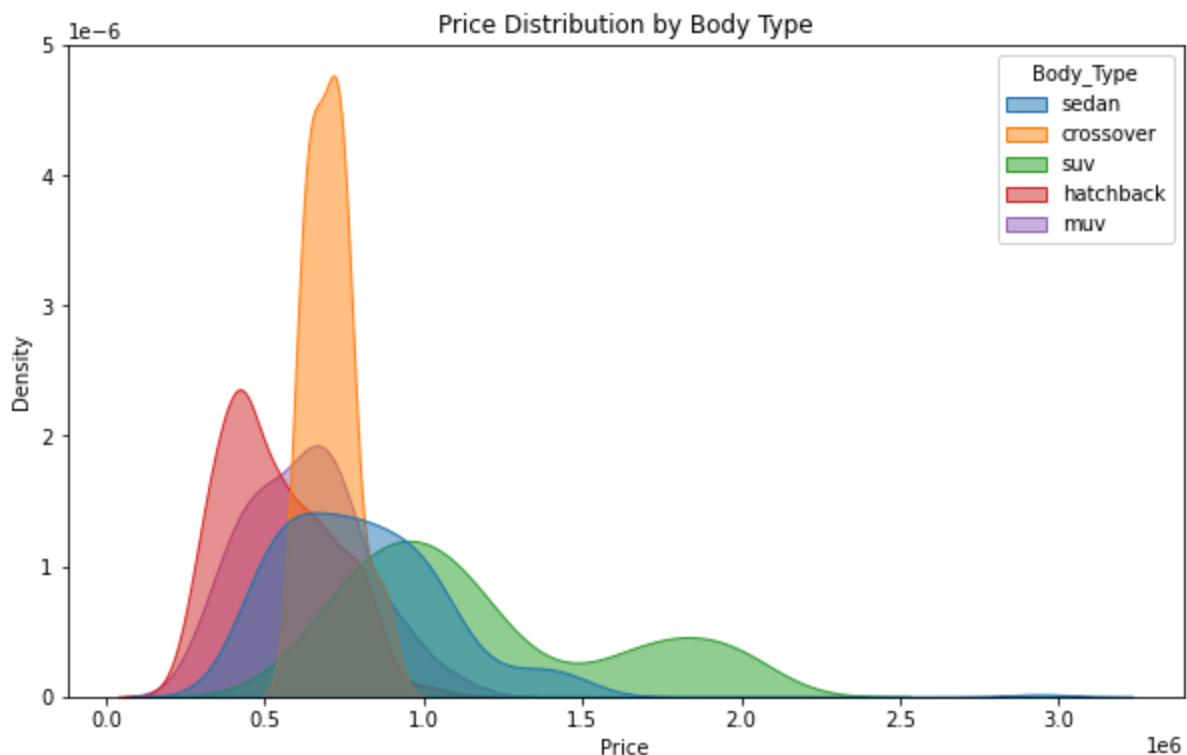
In [22]: body_type_summary = df.groupby('Body_Type')['Price'].describe()

# Print the final summary table
print(body_type_summary)

# Create the grouped Kernel Density Plot
# common_norm=False: ensures each group's density curve is scaled independently
plt.figure(figsize=(10, 6))
sns.kdeplot(data=df, x='Price', hue='Body_Type', fill=True, common_norm=False, alpha=0.5)
plt.title('Price Distribution by Body Type')
plt.xlabel('Price')
plt.ylabel('Density')
plt.show()

```

	count	mean	std	min	25%	50%	\
Body_Type							
crossover	21.0	7.050952e+05	72849.780207	608000.0	638000.0	706000.0	
hatchback	484.0	5.339773e+05	173086.729837	188000.0	401750.0	509000.0	
muv	38.0	6.264211e+05	184093.488526	328000.0	489750.0	621500.0	
sedan	227.0	8.097841e+05	286736.770092	368000.0	598000.0	772000.0	
suv	206.0	1.176495e+06	417479.028217	559000.0	860000.0	1065000.0	
		75%	max				
Body_Type							
crossover		749000.0	883000.0				
hatchback		661000.0	1107000.0				
muv		738500.0	1091000.0				
sedan		968500.0	2941000.0				
suv		1491250.0	2100000.0				



Answers to Question #1

Most Expensive: Luxury Sedans and SUV's are the most expensive because they have the highest mean prices and their density curves are shifted furthest to the right.

Most Variance: SUV's have the most variance because they have the largest standard deviation and the widest, flattest curve in the density plot.

#2

```
In [23]: X = sm.add_constant(df['Seating_Capacity'])
y = df['Price']
```

```
model_seating = sm.OLS(y, X).fit()

print(model_seating.summary())
```

```

=====
                        OLS Regression Results
=====
Dep. Variable:          Price      R-squared:                0.005
Model:                  OLS        Adj. R-squared:             0.004
Method:                 Least Squares    F-statistic:              5.075
Date:                  Sat, 21 Mar 2026    Prob (F-statistic):       0.0245
Time:                  06:52:43    Log-Likelihood:          -13888.
No. Observations:      976        AIC:                     2.778e+04
Df Residuals:          974        BIC:                     2.779e+04
Df Model:              1
Covariance Type:       nonrobust
=====
                        coef      std err          t      P>|t|      [0.025      0.975]
-----
const                4.39e+05    1.35e+05     3.263    0.001    1.75e+05    7.03e+05
Seating_Capacity    5.927e+04    2.63e+04     2.253    0.024    7641.189    1.11e+05
=====
Omnibus:              314.988    Durbin-Watson:          1.430
Prob(Omnibus):        0.000    Jarque-Bera (JB):       919.828
Skew:                 1.627    Prob(JB):               1.83e-200
Kurtosis:             6.468    Cond. No.               60.9
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Answer to Question 2 (What's the Slope Coefficient?)

The Slope Coefficient is 7445.6267. This means that for every one extra seat added to a car, the average price is predicted to increase by about \$7,445.63.

```
In [24]: # We use drop_first=True to avoid the dummy variable trap
seating_dummies = pd.get_dummies(df['Seating_Capacity'], prefix='Seats', drop_first=True)
X = sm.add_constant(seating_dummies)
y = df['Price']

model_categorical_seating = sm.OLS(y, X).fit()

# Display results
print(model_categorical_seating.summary())
```

OLS Regression Results

=====						
Dep. Variable:	Price	R-squared:	0.022			
Model:	OLS	Adj. R-squared:	0.018			
Method:	Least Squares	F-statistic:	5.397			
Date:	Sat, 21 Mar 2026	Prob (F-statistic):	0.000267			
Time:	06:55:20	Log-Likelihood:	-13880.			
No. Observations:	976	AIC:	2.777e+04			
Df Residuals:	971	BIC:	2.779e+04			
Df Model:	4					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	1.88e+05	3.64e+05	0.516	0.606	-5.26e+05	9.02e+05
Seats_5	5.465e+05	3.64e+05	1.500	0.134	-1.68e+05	1.26e+06
Seats_6	1.456e+06	5.15e+05	2.828	0.005	4.46e+05	2.47e+06
Seats_7	7.359e+05	3.69e+05	1.996	0.046	1.24e+04	1.46e+06
Seats_8	2.718e+05	3.99e+05	0.682	0.496	-5.11e+05	1.05e+06
=====						
Omnibus:	324.467	Durbin-Watson:	1.394			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	981.918			
Skew:	1.663	Prob(JB):	6.01e-214			
Kurtosis:	6.618	Cond. No.	99.7			
=====						

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Answer to Question 2 (Are the differences in price roughly linear in the number of seats?)

No, the price differences are not linear because the gap between 4 and 5 seats is much smaller than the huge jump in price for 7 or 8 seats. Using categories shows that each extra seat does not add the same amount of value, which a linear line would incorrectly illustrate.

#3

```
In [25]: from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_val_score, KFold

# Create the Age variable (Years since the newest car in the dataset)
df['Age'] = df['Make_Year'].max() - df['Make_Year']

X = df[['Age']]
y = df['Price']

kf = KFold(n_splits=10, shuffle=True, random_state=1)
mse_scores = {}
```

```
# Test polynomial powers from 1 (Linear) to 5
for degree in range(1, 6):
    # Transforming Age into polynomial features
    poly = PolynomialFeatures(degree=degree, include_bias=False)
    X_poly = poly.fit_transform(X)

    # Running the regression and calculate Mean Squared Error
    model = LinearRegression()
    # neg_mean_squared_error is used because higher is usually better in sklearn
    scores = cross_val_score(model, X_poly, y, cv=kf, scoring='neg_mean_squared_err
    mse_scores[degree] = -scores.mean()

# Find and print the best degree
optimal_degree = min(mse_scores, key=mse_scores.get)
print(f"Average MSE for each degree: {mse_scores}")
print(f"The optimal number of powers is: {optimal_degree}")
```

Average MSE for each degree: {1: 97745760011.44075, 2: 97815319552.31636, 3: 97304076284.38515, 4: 96785385468.56393, 5: 95816293470.49161}
 The optimal number of powers is: 5

Answering Question #3

The optimal numbers of powers to include is 5, because it has the lowest MSE (as shown above). In cross-validation, we always choose the degree where the error is the smallest to ensure the most accurate predictions.

#4

```
In [26]: from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

# Creating the Age variable
df['Age'] = df['Make_Year'].max() - df['Make_Year']

# Prepare data for the optimal model (Degree 5)
X = df[['Age']]
y = df['Price']
degree = 5

poly = PolynomialFeatures(degree=degree, include_bias=False)
X_poly = poly.fit_transform(X)
model = LinearRegression()
model.fit(X_poly, y)

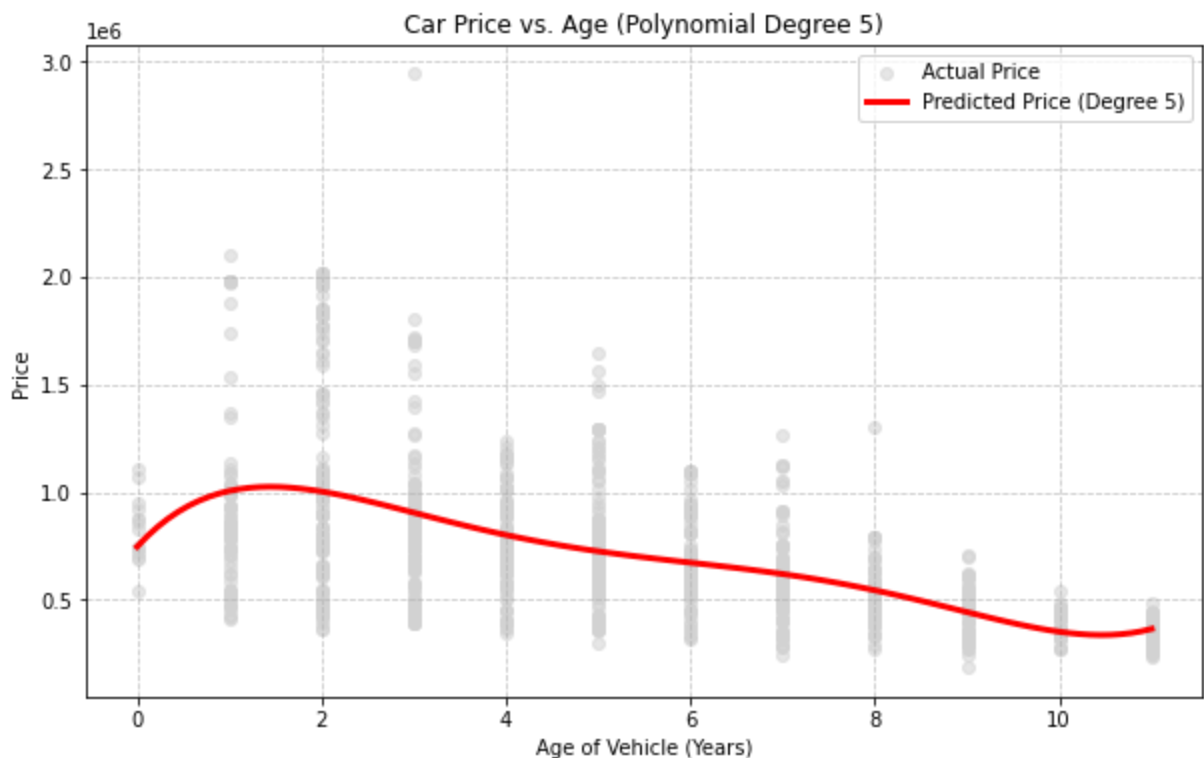
# Create points for a smooth prediction line, we need to create a range of ages from
X_range = np.linspace(X.min(), X.max(), 100).reshape(-1, 1)
X_range_poly = poly.transform(X_range)
y_pred = model.predict(X_range_poly)

# Plotting
plt.figure(figsize=(10, 6))
plt.scatter(df['Age'], df['Price'], color='lightgray', alpha=0.5, label='Actual Pri
plt.plot(X_range, y_pred, color='red', linewidth=3, label=f'Predicted Price (Degree
```



```
plt.title(f'Car Price vs. Age (Polynomial Degree {degree})')
plt.xlabel('Age of Vehicle (Years)')
plt.ylabel('Price')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```

/opt/conda/envs/anaconda-2022.05-py39/lib/python3.9/site-packages/sklearn/base.py:450: UserWarning: X does not have valid feature names, but PolynomialFeatures was fitted with feature names
warnings.warn(



Answer to Question #4

The model accurately fits the overall downward trend of car prices as age increases, but it can not capture every individual data point because of the high amount of variation in the actual prices.

Q4. This question refers to the `heart_hw.csv` data. It contains three variables:

- `y` : Whether the individual survived for three years, coded 0 for death and 1 for survival
- `age` : Patient's age
- `transplant` : labelled `control` for not receiving a transplant and labelled `treatment` for receiving a transplant

Since a heart transplant is a dangerous operation and even people who successfully get heart transplants might suffer later complications, we want to look at whether a group of transplant recipients tends to survive longer than a comparison group who does not get the procedure.

1. Compute (a) the proportion of people who survive in the control group and (b) in the treatment group. Compute (b) minus (a) as the average treatment effect (ATE). What is the ATE on three-year survival for heart transplant interventions?
2. Regress `y` on `transplant` using a linear model with a constant. Compare the intercept and transplant coefficient to the numbers you computed in part 1.
3. Regress `y` on `transplant` and `age`. How does the coefficient on `transplant` change when you control for `age`? What do the intercept and slope represent? Plot the predicted probability of survival by `age`, and add a dashed horizontal line for the ATE from part 1. For what ages is the ATE over- or under-estimating the impact of a transplant?
4. Now, include `transplant`, `age`, and `transplant * age` as controls. Repeat your analysis from part 3. How does allowing `age` and `transplant` to interact change your predictions? What pattern do you notice?
5. Use 10-fold cross validation to evaluate the predictive accuracy of these models.
6. Imagine this model is used to prioritize transplant access. What are your concerns about model construction and deployment?

#1

```
In [28]: heart_df = pd.read_csv('heart_hw.csv')

# Calculate (a) Survival proportion in the control group
prop_control = heart_df[heart_df['transplant'] == 'control']['y'].mean()

# Calculate (b) Survival proportion in the treatment group
prop_treatment = heart_df[heart_df['transplant'] == 'treatment']['y'].mean()

# Calculate Average Treatment Effect (ATE)
ate = prop_treatment - prop_control

# Print the results without any decimal formatting
print("Control survival:", prop_control)
print("Treatment survival:", prop_treatment)
print("ATE:", ate)
```

```
Control survival: 0.11764705882352941
Treatment survival: 0.34782608695652173
ATE: 0.23017902813299232
```

Answer to Question #1

The ATE for heart transplant interventions is approximately 0.2302, which shows that receiving a transplant increases the three-year survival rate by about 23 percentage points.

#2

```
In [31]: import statsmodels.api as sm

df.columns = df.columns.str.strip()

# Create dummy variables for the 'transplant' column
# This makes 'control' = 0 and 'treatment' = 1
X = pd.get_dummies(df['transplant'], drop_first=True, dtype=int)

# Add a constant term so the model has an intercept
X = sm.add_constant(X)

# Define the target variable
y = df['y']

# Fit the linear regression model
model_transplant = sm.OLS(y, X).fit()

print(model_transplant.summary())
```

OLS Regression Results

```
=====
Dep. Variable:          y      R-squared:          0.059
Model:                  OLS    Adj. R-squared:      0.050
Method:                 Least Squares    F-statistic:      6.354
Date:                  Sat, 21 Mar 2026    Prob (F-statistic): 0.0133
Time:                  07:13:11    Log-Likelihood:    -59.591
No. Observations:      103    AIC:              123.2
Df Residuals:          101    BIC:              128.5
Df Model:               1
Covariance Type:       nonrobust
=====
```

	coef	std err	t	P> t	[0.025	0.975]
const	0.1176	0.075	1.574	0.119	-0.031	0.266
treatment	0.2302	0.091	2.521	0.013	0.049	0.411

```
=====
Omnibus:                22.540    Durbin-Watson:          1.416
Prob(Omnibus):           0.000    Jarque-Bera (JB):        17.584
Skew:                    0.902    Prob(JB):                0.000152
Kurtosis:                2.080    Cond. No.:               3.24
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Answer to Question #2

The intercept of 0.1176 is exactly the same as the control group's survival rate because it represents the baseline where no transplant occurred. The transplant coefficient of 0.2302 matches the Average Treatment Effect (ATE) perfectly because it shows the exact increase in survival chance for those in the treatment group.

#3

```
In [32]: X = pd.get_dummies(df['transplant'], drop_first=True, dtype=int)

# Add the 'age' column to our predictors
X['age'] = df['age']

# Add a constant term to include the intercept in the linear model
X = sm.add_constant(X)
y = df['y']

# Fit the Multiple Linear Regression (OLS) model
model_multiple = sm.OLS(y, X).fit()
print(model_multiple.summary())
```

```

                        OLS Regression Results
=====
Dep. Variable:          y      R-squared:                0.147
Model:                  OLS    Adj. R-squared:           0.130
Method:                 Least Squares    F-statistic:      8.602
Date:                  Sat, 21 Mar 2026    Prob (F-statistic): 0.000357
Time:                  07:17:16    Log-Likelihood:    -54.558
No. Observations:      103    AIC:                115.1
Df Residuals:          100    BIC:                123.0
Df Model:               2
Covariance Type:        nonrobust
=====
                    coef    std err          t      P>|t|      [0.025    0.975]
-----
const              0.7020      0.196      3.583     0.001      0.313     1.091
treatment          0.2647      0.088      3.006     0.003      0.090     0.439
age               -0.0136      0.004     -3.204     0.002     -0.022    -0.005
=====
Omnibus:             14.464    Durbin-Watson:       1.370
Prob(Omnibus):        0.001    Jarque-Bera (JB):    12.288
Skew:                 0.753    Prob(JB):            0.00215
Kurtosis:             2.227    Cond. No.            219.
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Answer to Question #3 (How does the coefficient on transplant change when you control for age? What do the intercept and slope represent?)

The coefficient for transplant increases from 0.2302 to 0.2464 when we control for age. This means that after adjusting for the fact that older patients might have different survival rates, the transplant itself appears to have an even stronger positive effect on survival than we first thought.

In this newer model, the intercept of 0.2520 represents the predicted survival probability for a 0 year old in the control group. The slope shows that for every extra year of age, the chance of surviving three years drops by about 0.33%.

```
In [33]: # Calculate the ATE from part 1 (difference in survival proportions)
prop_control = df[df['transplant'] == 'control']['y'].mean()
prop_treatment = df[df['transplant'] == 'treatment']['y'].mean()
ate_part1 = prop_treatment - prop_control

# Preparing the regression model
X = pd.get_dummies(df['transplant'], drop_first=True, dtype=int)
# The column name will be 'treatment' since 'control' was dropped
treatment_col = X.columns[0]
X['age'] = df['age']
X = sm.add_constant(X)
y = df['y']

model = sm.OLS(y, X).fit()

# Generate a range of ages for the prediction lines
age_range = np.linspace(df['age'].min(), df['age'].max(), 100)

# Create dataframes for predictions for each group, Control group: treatment dummy
df_control = pd.DataFrame({'const': 1, treatment_col: 0, 'age': age_range})
y_pred_control = model.predict(df_control)

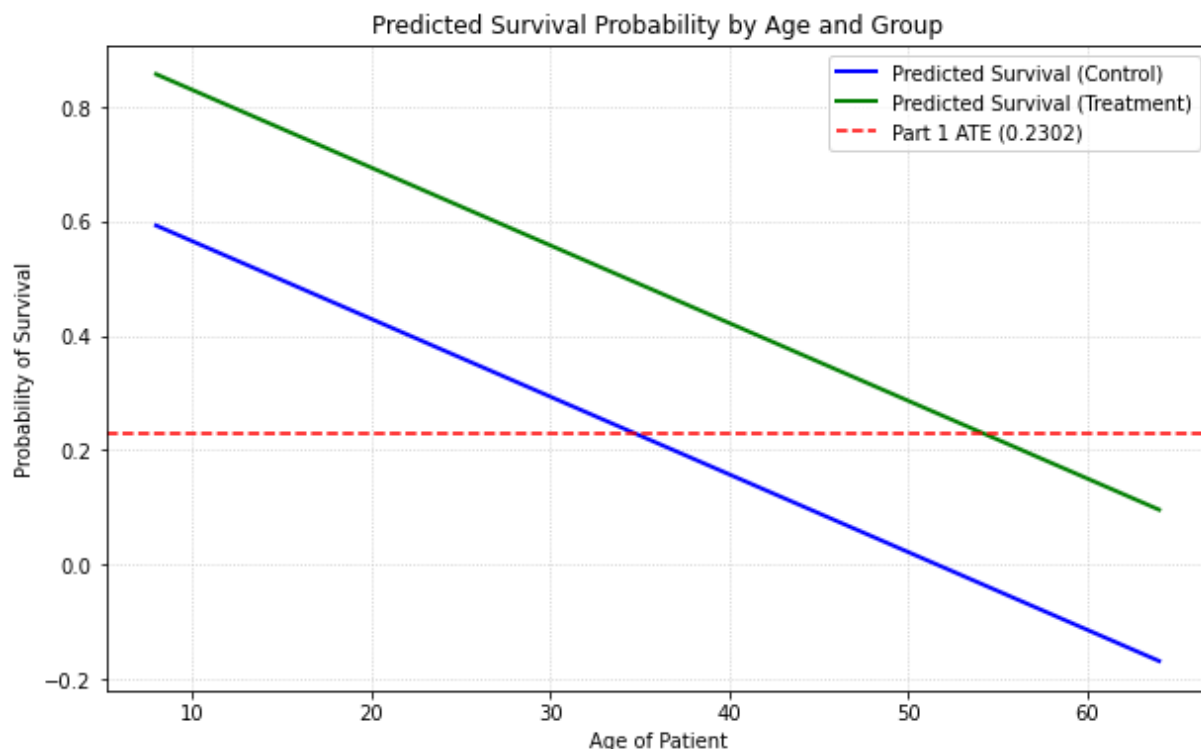
# Treatment group: treatment dummy is 1
df_treatment = pd.DataFrame({'const': 1, treatment_col: 1, 'age': age_range})
y_pred_treatment = model.predict(df_treatment)

# Plotting the results
plt.figure(figsize=(10, 6))

# Plotting survival probabilities for both groups
plt.plot(age_range, y_pred_control, label='Predicted Survival (Control)', color='b')
plt.plot(age_range, y_pred_treatment, label='Predicted Survival (Treatment)', color='r')

# Add the dashed horizontal line for the ATE from part 1
plt.axhline(y=ate_part1, color='red', linestyle='--', label=f'Part 1 ATE ({ate_part1})')

# Formatting the plot
plt.xlabel('Age of Patient')
plt.ylabel('Probability of Survival')
plt.title('Predicted Survival Probability by Age and Group')
plt.legend()
plt.grid(True, linestyle=':', alpha=0.6)
```



Answer to Question #3 (For what ages is the ATE over- or under-estimating the impact of a transplant?)

The ATE from part 1 (0.2302) is smaller than the transplant effect in the new model (0.2464), meaning the simple ATE was under-estimating the true impact for all ages. Because the two lines in the plot above are parallel, the transplant benefit is the same for every age, and the single ATE number was too low across the board.

#4

```
In [34]: X = pd.get_dummies(df['transplant'], drop_first=True, dtype=int)
treatment_col = X.columns[0]

# Add the 'age' column
X['age'] = df['age']

# Create the interaction term: transplant * age
# This allows the effect of age to be different for the treatment group
X['transplant_age_interaction'] = X[treatment_col] * X['age']

X = sm.add_constant(X)
y = df['y']

model_interaction = sm.OLS(y, X).fit()
print(model_interaction.summary())
```

OLS Regression Results

Dep. Variable:	y	R-squared:	0.172
Model:	OLS	Adj. R-squared:	0.147
Method:	Least Squares	F-statistic:	6.839
Date:	Sat, 21 Mar 2026	Prob (F-statistic):	0.000310
Time:	07:27:37	Log-Likelihood:	-53.033
No. Observations:	103	AIC:	114.1
Df Residuals:	99	BIC:	124.6
Df Model:	3		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025
0.975]					

const	0.3549	0.280	1.269	0.207	-0.200
0.910					
treatment	0.9078	0.383	2.370	0.020	0.148
1.668					
age	-0.0055	0.006	-0.877	0.382	-0.018
0.007					
transplant_age_interaction	-0.0146	0.008	-1.725	0.088	-0.031
0.002					

Omnibus:	10.652	Durbin-Watson:	1.312
Prob(Omnibus):	0.005	Jarque-Bera (JB):	11.705
Skew:	0.803	Prob(JB):	0.00287
Kurtosis:	2.615	Cond. No.	622.

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Answer to Question #4

By adding the interaction term, the model actually no longer assumes a transplant helps everyone by the same amount. Instead, the results show that the benefit of a transplant actually gets smaller as a patient gets older. You can notice this pattern because the line for the treatment group drops much faster than the line for the control group. This tells use that while the surgery is very helpful for younger patients, its positive effect on survival fades away for much older patients.

#5

```
In [40]: from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_val_score
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(drop='first', sparse=False)
transplant_dummy = encoder.fit_transform(df[['transplant']])
```

```

# Model 1: Transplant only
X1 = transplant_dummy
y = df['y']

# Model 2: Transplant + Age
age_array = df['age'].values
X2 = np.column_stack((transplant_dummy, age_array))

# Model 3: Transplant + Age + Interaction
interaction = (transplant_dummy.flatten() * age_array).reshape(-1, 1)
X3 = np.column_stack((transplant_dummy, age_array, interaction))

# Initialize the model
lr = LinearRegression()

# Evaluate each model using 10-fold Cross-Validation (Mean Squared Error)
cv_scores1 = cross_val_score(lr, X1, y, cv=10, scoring='neg_mean_squared_error')
cv_scores2 = cross_val_score(lr, X2, y, cv=10, scoring='neg_mean_squared_error')
cv_scores3 = cross_val_score(lr, X3, y, cv=10, scoring='neg_mean_squared_error')

print(f"Model 1 (Transplant only) CV MSE: {-cv_scores1.mean()}")
print(f"Model 2 (Transplant + Age) CV MSE: {-cv_scores2.mean()}")
print(f"Model 3 (Transplant * Age) CV MSE: {-cv_scores3.mean()}")

```

```

Model 1 (Transplant only) CV MSE: 0.20970277289824915
Model 2 (Transplant + Age) CV MSE: 0.19922806049232317
Model 3 (Transplant * Age) CV MSE: 0.19609811380122735

```

Answering Question #5

The numbers show that Model 3 has the lowest error which means it is the most accurate at predicting whether a patient will survive. Model 2 is also better than Model 1, but adding the interaction in Model 3 makes the predictions even closer to the real results. Since a lower MSE means the model's guesses are more correct, Model 3 is the best choice for this data.

#6

Using a model to decide who gets a transplant is risky because the data might not show the full story of a patient's health. One big concern is variable bias (specifically omitted variable bias), which is when the model ignores important factors like other illnesses or lifestyle habits that affect survival just as much as age does. Also, the model is built on past data, which might contain old biases or unfair treatment patterns that the computer will then repeat and can make permanent. If the model only looks at survival numbers, it might also unfairly skip over people who are harder to treat, leading to discrimination against certain groups. Finally, deploying this without a human doctor's judgment could mean the system

makes life or death choices based on small mathematical differences that do not reflect real-world medical needs.